

Homework 5: Fine-Tuning FLAN-T5-Small for Dialogue Summarization Using LoRA

Naga Sai Haritha Gottumukkala
Department of Applied Data Science
San Jose State University

TABLE 1 PERSONAL PARAMETERS

Parameter	Value
SID4	1598
SEED	1598
SLICE	598
HP_ID	2
CLS_A	8
CLS_B	5

Abstract

This assignment investigates dialogue summarization by fine-tuning FLAN-T5-small using Low-Rank Adaptation (LoRA). The experiment used 1,999 training examples and 499 validation examples from the assigned DialogSum dataset. Baseline summaries were generated for two fixed test dialogues, followed by separate LoRA experiments with ranks 4 and 16. Both models were trained for two epochs on a Tesla T4 GPU. Rank 4 trained 172,032 parameters and achieved a validation loss of 1.4146. Rank 16 trained 688,128 parameters and achieved a validation loss of 1.4198. Fine-tuning improved the coverage of some important details, but speaker-role errors and repetition

remained. Increasing the rank did not improve summary quality on the two inspected examples.

Index Terms—LoRA, PEFT, FLAN-T5, DialogSum, dialogue summarization.

I. INTRODUCTION

Summarizing a conversation requires more than selecting a few sentences. A useful summary should explain what happened, identify who did what, and leave out details that are not important. This becomes difficult when several speakers discuss different actions or repeat similar information.

In this assignment, I used the pretrained model google/flan-t5-small for dialogue summarization. I first checked its outputs before training so that I had a baseline. I then fine-tuned the model using LoRA and compared the new summaries with the original outputs and the human-written reference summaries.

The assignment also required an experiment with two LoRA ranks: 4 and 16. My aim was to check whether a larger adapter would produce better summaries and whether any improvement justified the additional trainable parameters.

I kept the training data, number of epochs, learning rate, and generation settings the same for both experiments. Each experiment

started from a fresh copy of the pretrained model.

II. MODEL AND FINE-TUNING APPROACH

A. FLAN-T5-Small

FLAN-T5-small is a text-to-text model. It accepts text as its input and generates text as its output. For this assignment, the input was a conversation with a short summarization instruction, and the expected output was a summary.

The model uses an encoder-decoder architecture. The encoder processes the input dialogue and builds representations of its content. The decoder uses these representations to generate the summary one token at a time.

During training, the reference summary provides the target tokens. The model learns by comparing its predicted token probabilities with those targets. During inference, the reference summary is not supplied to the model; it must generate a summary from the dialogue alone.

B. Why I Used LoRA

Updating every parameter of a pretrained model can require considerable memory and computation. LoRA provides a way to adapt the model while updating a much smaller set of parameters.

For a selected weight matrix, LoRA learns an adjustment using two smaller matrices. The original pretrained weights remain frozen. The rank determines the inner dimension of these adapter matrices and therefore affects the number of trainable parameters.

A larger rank gives the adapter more capacity to learn adjustments. However, greater capacity does not automatically produce better summaries. The result also depends on the data, training settings, and the behavior of the base model.

In this experiment, I attached LoRA adapters to the attention query and value projections, named q and v. Query projections help form the attention scores used to relate tokens, while value projections provide the information combined through attention.

III. PERSONAL PARAMETERS AND REPRODUCIBILITY

I used the following personal parameters:

Parameter	Value
SID4	1598
SEED	1598
SLICE	598
HP_ID	2
CLS_A	8
CLS_B	5

I explicitly set the Python, NumPy, and PyTorch random seeds to 1598. I also supplied this seed to the training configuration and used it to select the two test examples.

HW5 requires the same rank comparison for all students, so I trained both rank 4 and rank 16. The assignment did not specify an HP_ID-based learning-rate mapping or require the use of SLICE and the focus-class parameters.

The experiment ran in Google Colab with the environment shown in Table I.

TABLE I RECORDED SOFTWARE AND HARDWARE ENVIRONMENT

Component	Recorded value
GPU	Tesla T4
Reported GPU memory	14.56 GB
PyTorch	2.11.0+cu128
CUDA	12.8
Transformers	5.16.1
Datasets	4.8.5
PEFT	0.20.0
Accelerate	1.14.0
Training precision	FP32

The setup cell removed torchao, and a later check confirmed that it was not available in the environment used for this run. I requested deterministic algorithms using `torch.use_deterministic_algorithms(True, warn_only=True)`. However, training produced a warning that a memory-efficient attention operation was nondeterministic. Because warning-only mode allows the operation to continue, the fixed seed does not guarantee exactly identical numerical results in another run.

IV. DATA PREPARATION

A. Loading and Inspecting the Dataset

I loaded the assigned dataset using:
`load_dataset("neil-code/dialogsum-test")`
The dataset contained the splits shown in Table II.

TABLE II DATASET SPLITS

Split	Examples	Purpose
Training	1,999	Learning the LoRA adapter weights
Validation	499	Monitoring loss and selecting the best checkpoint
Test	499	Selecting the two fixed inference examples

The original columns were id, dialogue, summary, and topic. I used the dialogue as the input and the summary as the target. I used all the training and validation examples available in this dataset version.

The two examples used for output comparison came from the test split.

B. Preparing the Input and Target

For each example, I created two fields:

- `input_text`: the instruction followed by the dialogue.
- `target_text`: the original human-written summary.

The instruction was:

“Summarize the following dialogue:”

Adding this instruction made the requested task explicit. I used the same input format during training and inference.

C. Two Processed Samples

The following examples show the processed dialogue-to-summary format used in the notebook.

Processed Sample 1

Input:

Summarize the following dialogue:

#Person1#: Hi, Mr. Smith. I'm Doctor Hawkins. Why are you here today?

#Person2#: I found it would be a good idea to get a check-up.

#Person1#: Yes, well, you haven't had one for 5 years. You should have one every year.

#Person2#: I know. I figure as long as there is nothing wrong, why go see the doctor?

#Person1#: Well, the best way to avoid serious illnesses is to find out about them early. So try to come at least once a year for your own good.

#Person2#: Ok.

#Person1#: Let me see here. Your eyes and ears look fine. Take a deep breath, please.

Do you smoke, Mr. Smith?

#Person2#: Yes.

#Person1#: Smoking is the leading cause of lung cancer and heart disease, you know. You really should quit.

#Person2#: I've tried hundreds of times, but I just can't seem to kick the habit.

#Person1#: Well, we have classes and some medications that might help. I'll give you more information before you leave.

#Person2#: Ok, thanks doctor.

Target:

“Mr. Smith's getting a check-up, and Doctor Hawkins advises him to have one every year. Hawkins'll give some information about their classes and medications to help Mr. Smith quit smoking.”

Processed Sample 2

Input:

Summarize the following dialogue:

#Person1#: Hello Mrs. Parker, how have you been?

#Person2#: Hello Dr. Peters. Just fine thank you. Ricky and I are here for his vaccines.

#Person1#: Very well. Let's see, according to his vaccination record, Ricky has received his Polio, Tetanus and Hepatitis B shots. He is 14 months old, so he is due for Hepatitis A, Chickenpox and Measles shots.

#Person2#: What about Rubella and Mumps?

#Person1#: Well, I can only give him these for now, and after a couple of weeks I can administer the rest.

#Person2#: OK, great. Doctor, I think I also may need a Tetanus booster. Last time I got it was maybe fifteen years ago!

#Person1#: We will check our records and I'll have the nurse administer and the booster as well. Now, please hold Ricky's arm tight, this may sting a little.

Target:

“Mrs Parker takes Ricky for his vaccines. Dr. Peters checks the record and then gives Ricky a vaccine.”

These samples confirmed that each processed input contained the instruction and dialogue, while its target contained only the summary.

D. Tokenization and Batch Preparation

I tokenized the inputs with a maximum length of 512 tokens and the targets with a maximum length of 128 tokens. Longer sequences were truncated.

The tokenized examples contained input_ids, attention_mask, and labels. The input IDs represented the input tokens, the attention mask identified valid input

positions, and the labels represented the target-summary tokens.

I used DataCollatorForSeq2Seq to prepare batches with dynamic padding. This allowed examples of different lengths to be combined into a batch.

As a basic check, I decoded the first tokenized target and compared it with the original summary. The notebook showed that this example contained 291 input tokens and 48 target tokens.

V. BASELINE INFERENCE

Before applying LoRA, I loaded the original FLAN-T5-small model and generated summaries for two selected test examples. Using seed 1598, the selected test indices were 240 and 216. I saved these examples and reused them for both fine-tuned models. Table III shows the generation settings.

TABLE III

GENERATION SETTINGS

Setting	Value
Maximum input Length	512 tokens
Maximum new tokens	80
Number of beams	4
Sampling	Disabled
Model mode	Evaluation

Beam search maintains several candidate sequences during generation. Disabling sampling avoids random token sampling during decoding. I also used torch.no_grad() because inference did not require gradient calculations.

The baseline summaries were saved in baseline_outputs.json. Their exact text is included in Section VIII alongside the fine-tuned outputs.

VI. LORA CONFIGURATION

I used PEFT to configure and attach the adapters. Table IV shows the settings for both experiments.

Setting	Rank 4 Experiment	Rank 16 Experiment
Rank	4	16
Alpha	16	16
Dropout	0.05	0.05
Target Modules	q,v	q,v
Task Type	SEQ_2_SE Q_LM	SEQ_2_SE Q_LM
Bias	None	None

The rank controls adapter size. Alpha controls the scaling of the learned update, and dropout provides regularization within the LoRA path during training.

I loaded a fresh base model for each experiment. This ensured that rank 16 started from the original pretrained weights rather than from the rank 4 fine-tuned model.

After attaching the adapters, I counted the total parameters and the parameters with requires_grad=True.

TABLE V

TOTAL AND TRAINABLE PARAMETERS

Metric	Rank 4	Rank 16
--------	--------	---------

Total parameters	77,133,184	77,649,280
Trainable parameters	172,032	688,128
Trainable percentage	0.2230%	0.8862%

The larger adapter had exactly four times as many trainable parameters. Even so, both experiments trained less than 1% of the total parameters.

I also printed the names of several trainable parameters. These included the lora_A and lora_B matrices attached to the query and value modules.

VII. FINE-TUNING AND TRAINING RESULTS

A. Training Configuration

Both models used the settings in Table VI.

Settings	Value
Epochs	2
Training Batch Size	8
Evaluation Batch Size	8
Gradient accumulation steps	2
Learning Rate	0.0002
Weight Decay	0.01
Maximum Gradient Norm	1.0
Precision	FP32
Evaluation Strategy	Every Epoch
Checkpoint Saving	Every Epoch

Logging Interval	Every 25 steps
CheckPoint Selection	Lowest Validation Loss

Gradient accumulation combined gradients from two mini-batches before an optimizer update. On the single GPU, this gave a nominal effective batch size of 16 examples. Gradient clipping limited the gradient norm to 1.0. The trainer evaluated the model after each epoch and was configured to reload the checkpoint with the lowest validation loss.

B. Pre-Training Sanity Check

Before each training run, I passed the first two training examples through the model and calculated the loss. Both ranks produced a loss of 2.2771.

This confirmed that the inputs, labels, and forward pass produced a valid numerical result. It was only a two-example check and should not be treated as the baseline loss for the entire validation set.

C. Training Logs

The displayed epoch-boundary results were as follows.

Rank	Epoch	logged Training	Validation
4	1	3.352018	1.437019
4	2	3.206309	1.414560
16	1	3.376159	1.446143
16	2	3.217955	1.419776

Both models showed lower validation loss after the second epoch. The displayed training losses also decreased.

These training-loss entries are the values reported at the logging points. They are not

the average losses over the complete training run.

D. Final Recorded Metrics

TABLE VIII FINAL EXPERIMENT RESULTS

Metrics	Rank 4	Rank 16
Average Training Loss	3.4843	3.4987
Validation Loss	1.41461	1.4198
Global Training Steps	250	250
Training time	2.70 minutes	2.79 minutes

Rank 4 had a slightly lower validation loss, with a difference of approximately 0.0052. The training-time difference was also small. Training and validation losses are measured under different conditions, including training versus evaluation behavior. Their numerical difference alone does not prove a problem. Similarly, the two-example sanity-check loss should not be directly compared with the average training loss.

E. Saving the Trained Adapters

The training calls were:

```
train_result_r4 = trainer_r4.train()
```

```
train_result_r16 = trainer_r16.train()
```

After training, I saved the adapters and tokenizers in:

- dialogsum_lora_r4_adapter/
- dialogsum_lora_r16_adapter/

These folders contain the adapter artifacts. The corresponding FLAN-T5-small base model is still required when loading an adapter for inference.

VIII. POST-TRAINING INFERENCE AND OUTPUT COMPARISON

I generated summaries again for test indices 240 and 216 using the same input formatting and generation settings.

The outputs below are copied exactly from the notebook. I have kept the repetition and wording because they are part of the model's actual behavior.

A. Example 1 — Car Accident

Test index: 240

The dialogue describes a customer calling ABC Rent-a-car Company after an accident near the border. The customer is safe, but the customer's friend is seriously injured. The company representative agrees to call an ambulance and the police.

Reference summary:

"#Person1# rent a car from ABC Rent-a-car Company this morning and met an accident. #Person2# will call an ambulance and police for #Person1#."

Baseline summary:

"I ran into the guardrail and got into a car."

Rank 4 summary:

"#Person1# and #Person2# are on the way to Niagara Falls. #Person1#'s friend is seriously injured and #Person2# will call an ambulance and the police."

Rank 16 summary:

"#Person1# and #Person2# are on the way to Niagara Falls. They have a car accident near the border. #Person1# will call an ambulance and the police."

Comparison and explanation:

The baseline was too short to explain the main event. It left out the injured friend and the request for emergency help.

Rank 4 included both of these important details. It also correctly stated that Person2

would call the ambulance and police. However, it incorrectly described Person2 as travelling to Niagara Falls with Person1. In the dialogue, Person2 is the rental company representative answering the phone. Rank 16 included the accident location, but it made an additional mistake by saying that Person1 would call emergency services. This changed who was responsible for the action.

For this example, rank 4 captured more useful information than the baseline, but its summary was still not completely accurate.

B. Example 2 — Shopping for Poster Materials

Test index: 216

The dialogue concerns preparing posters for a group report. The speakers discuss making a shopping list and finding materials using the signs above the shelves.

Reference summary:

“#Person1# and #Person2# will do a group report, so they plan to buy something to make posters. They make a shopping list before they go and find items according to signs on shelves.”

Baseline summary:

“I need to do a group report tomorrow. I need to do a group report tomorrow. I need to go to the stationer to buy something for the posters. I need markers, color pencils, a pair of scissors, a pair of highlighter, rulers. I need markers, color pencils and highlighters. I need markers, color pencils and highlighters”

Rank 4 summary:

“#Person1# and #Person2# need to do a group report tomorrow. #Person1# needs markers, color pencils and highlighters and

pencils. #Person1# also needs markers, color pencils and highlighters and paper clips.”

Rank 16 summary:

“#Person1# and #Person2# need to do a group report tomorrow. #Person1# needs markers, color pencils and highlighters and pencils. #Person1# needs markers, color pencils and highlighters and pencils. #Person1# needs markers, color pencils and highlighters and pencils.”

Comparison and explanation:

The baseline repeated the opening statement and several supplies. It contained relevant words but did not combine them into a concise account of the conversation.

Rank 4 produced a shorter summary and used the speaker-label style found in the references. However, it still repeated the materials and left out the shopping-list preparation and the use of shelf signs.

Rank 16 repeated the same sentence about supplies three times. The larger adapter did not solve the repetition problem and produced a less useful result than rank 4 for this example.

C. Overall Before and After Comparison

Fine-tuning helped the model include important details in the accident example and adopt a summary style closer to the dataset. However, the change was not a complete improvement: the models still confused speakers, omitted key actions, and repeated phrases. These outputs show why the generated text needs to be inspected alongside the numerical results.

IX. FAILURE ANALYSIS

I observed three main types of failure.

TABLE IX OBSERVED OUTPUT ERRORS

Error category	Example	Explanation
Speaker-role confusion	Both fine-tuned accident summaries describe Person2 as travelling	The models did not correctly preserve the relationship between the caller and company representative
Incorrect action assignment	Rank 16 says Person1 will call emergency services	The summary assigns an action to the wrong speaker
Repetition and missing information	Shopping summaries repeat supplies and omit the list and shelf signs	The models focus on repeated item names instead of summarizing the main actions

The notebook outputs show these errors, but they do not establish their exact cause.

Possible areas to investigate include the training duration, decoding settings, and the model's ability to track speaker roles. These would require further experiments.

The current generation configuration did not explicitly include an additional repetition penalty or a no-repeat n-gram constraint. Such settings could be tested later, but I did not change them during this comparison.

X. COMPARISON OF RANK 4 AND RANK 16

Rank 16 increased the number of trainable parameters from 172,032 to 688,128.

Despite this increase, it did not achieve a lower validation loss or better summaries on the two inspected examples.

Rank 4 retained the correct emergency-call responsibility in the accident example and showed less repetition in the shopping example. Considering both the adapter size and observed outputs, rank 4 was the more efficient choice for this run.

However, there is an important limitation in the comparison. Standard LoRA scales its update using alpha divided by rank. With alpha fixed at 16, this factor was 4 for rank 4 and 1 for rank 16. Therefore, increasing rank changed both the adapter capacity and its scaling factor.

The experiment supports a limited conclusion: rank 16 did not improve the recorded results under these settings. It does not prove that a larger rank would perform worse with different settings or more extensive training.

XI. LIMITATIONS AND POSSIBLE IMPROVEMENTS

The qualitative evaluation used only two test dialogues. This was enough to demonstrate the required before-and-after comparison, but it was not enough to measure overall summarization quality.

The notebook did not report ROUGE scores, even though evaluation libraries were installed. Therefore, the findings are based on validation loss and manual inspection rather than a full test-set evaluation.

Each rank was trained only once. Repeating the experiments with several seeds would

help determine whether the small validation-loss difference is consistent. The input length limit was 512 tokens. Longer dialogues could lose information through truncation, but the notebook did not measure how often this happened. This should be checked before drawing conclusions about performance on longer conversations.

The environment also used package installation commands without fixed versions. The recorded versions document this run, but pinning them would make it easier to recreate the software environment. Possible follow-up work would include evaluating more test examples, checking factual consistency, testing repetition controls, and repeating the rank comparison under more controlled scaling settings. These are proposed improvements and were not part of the reported run.

XII. CONCLUSION

In this assignment, I prepared the DialogSum data, generated baseline summaries, attached LoRA adapters, and trained two models with ranks 4 and 16. I saved both adapters and compared their outputs on the same two test dialogues. Rank 4 achieved a validation loss of 1.4146 while training only 0.2230% of the model parameters. Rank 16 trained 0.8862% of the parameters and achieved a validation loss of 1.4198.

The main observation was that increasing adapter size did not automatically improve the summaries. Rank 4 recovered useful information, but both models still made factual mistakes and repeated content. This experiment helped me see the difference

between producing text that sounds like a summary and producing a summary that correctly represents the conversation.

XIII .REFERENCES

- [1] N. S. H. Gottumukkala, “DATA266_1598_hw5.ipynb,” executed Homework 5 notebook, DATA 266, 2026.
- [2] N. S. H. Gottumukkala, “METRICS.md” and “RUN_LOG.txt,” Homework 5 experiment records, 2026.
- [3] “DialogSum dataset,” Hugging Face. Available: <https://huggingface.co/datasets/neil-code/dialogsum-test>
- [4] “FLAN-T5-small,” Google, Hugging Face model repository. Available: <https://huggingface.co/google/flan-t5-small>